XPLOIT Vault System — Writeup

Binary Exploitation | Linux x86-64 | Challenge: chal

1. Initial Recon

Three commands were run before making any changes:

```
$ file chal
$ strings chal
$ objdump -d chal > disasm.txt
```

file: 64-bit PIE ELF, dynamically linked, not stripped — symbol names preserved, easy to reverse.

strings: Revealed the success string (VAULT SYSTEM CLEARED.), the unlock prompt (Vault Unlock Code:), the state file name (.vault_state), and many decoy strings (FAKE_PASSWORD_123, 1234) to ignore.

First run: chmod +x chal && ./chal creates .vault_state then exits. This file seeds the unlock key and must exist before running the patched binary.

```
$ chmod +x chal && ./chal
Initializing XPLOIT Vault System...
[SYS] kernel handshake initialised.
[VAULT] Cold start detected. Vault state file initialised.
[VAULT] Authentication suspended. Re-run system to resume.
```

2. Function Map

Function	Role	Relevant?
main (0x1b0e)	Calls all modules. After user_auth, calls puts('Terminating...') and exits. Never calls unlock_vault_sequence.	YES — missing call
emit_system_diagnostics (0x1752)	getpid() → g_pid_seed = (pid XOR pid>>8) AND 0xFF	YES — key input
check_vault_state (0x178b)	Opens .vault_state. If absent: creates from time(), exits. If present: reads 1 byte into g vault byte.	YES — key input
user_authentication_module (0x1936)	Prompts Operator ID. BUG: compares hardcoded 1 to 999 — always fails, always exits(1).	YES — has the bug
unlock_vault_sequence (0x19f2)	Computes key = (pid_seed XOR vault_byte) XOR strlen(argv[0]). Reads hex input. On match: VAULT CLEARED.	YES — final target
security_watchdog (0x18ef)	ptrace(PTRACE_TRACEME) — kills process if debugger attached. Analysis must be static.	YES — anti-debug
initialize_telemetry, verify_network_cert, init_secure_channel, scan_memory_integrity, load_crypto_module, compute_session_hash, destroy_secure_channel, omega_protocol_legacy	All produce outputs that are never used by any other function.	NO — all decoys

legacy_auth_v1, unlock_backup_vault	Never called from main.	NO — dead code

3. Changes Made

Patch 1 — Fix auth bug (file offset 0x1951)

Problem: Local variable initialised to 1, then compared to 999. Always fails → exit(1). Execution never returns to main.

Original: `c7 45 ac 01 00 00 00  movl $0x1, -0x54(%rbp)`

Fixed: `c7 45 ac e7 03 00 00  movl $0x3e7, -0x54(%rbp)  ; 0x3e7 = 999`

Why: Now 999 == 999 → auth succeeds → function returns to main normally. Only 3 bytes changed.

Patch 2 — Route main to unlock_vault_sequence (file offset 0x1b8e)

Problem: After user_auth returns, main calls puts('Terminating...') and exits. unlock_vault_sequence is never called.

Original: `e8 cd f5 ff ff  call puts@plt`

Fixed: `e8 5f fe ff ff  call unlock_vault_sequence`

Why: Replaces a cosmetic print with the call to the vault unlock function. Relative displacement = 0x19f2 - 0x1b93 = -0x1a1 → 5f fe ff ff. Only 4 bytes changed.

patch.py

```
import shutil
shutil.copy('chal', 'chal_patched')
with open('chal_patched', 'r+b') as f:
    # Patch 1
    f.seek(0x1951)
    assert f.read(7) == bytes([0xc7,0x45,0xac,0x01,0x00,0x00,0x00])
    f.seek(0x1951)
    f.write(bytes([0xc7,0x45,0xac,0xe7,0x03,0x00,0x00]))
    # Patch 2
    f.seek(0x1b8e)
    assert f.read(5) == bytes([0xe8,0xcd,0xf5,0xff,0xff])
    f.seek(0x1b8e)
    f.write(bytes([0xe8,0x5f,0xfe,0xff,0xff]))
print('Done — chal_patched created')
```

4. The Unlock Code

Formula (from disassembly of unlock_vault_sequence):

```
key = (g_pid_seed XOR g_vault_byte) XOR (strlen(argv[0]) AND 0xFF)
Input must be entered as a hex string (parsed with strtol base 16)
```

Variable	Source	Notes
g_pid_seed	emit_system_diagnostics: (pid XOR pid>>8) AND 0xFF	Changes every run
g_vault_byte	First byte of .vault_state: (time() AND 0xFF) XOR 0x3C	Fixed after first run

strlen(argv[0])	Length of invocation path, e.g. './chal_patched' = 14	Fixed

Example: `vault_byte=0x4f, pid=99 → pid_seed=0x63, argv_len=14=0x0e → key=0x22 → send '22'`

solve.py

```
import subprocess, os
os.chdir(os.path.dirname(os.path.abspath(__file__)))
with open('.vault_state','rb') as f:
    vault_byte = f.read(1)[0]
argv0 = './chal_patched'
argv0_len = len(argv0) & 0xFF
proc = subprocess.Popen([argv0], stdin=subprocess.PIPE, stdout=subprocess.PIPE)
pid = proc.pid
pid_seed = (pid ^ (pid >> 8)) & 0xFF
key = (pid_seed ^ vault_byte) ^ argv0_len
print(f'key = {key:#x}')
stdout, _ = proc.communicate(input=b'anyoperator\n' + f'{key:x}\n'.encode())
print(stdout.decode())
```

5. Full Command Workflow

```
# Step 1 — give permissions and create .vault_state
chmod +x chal
./chal

# Step 2 — apply patches
python3 patch.py
chmod +x chal_patched

# Step 3 — run solve script (auto-computes and sends the key)
python3 solve.py
```

6. Final Output

```
Initializing XPLOIT Vault System...
[SYS] kernel handshake initialised.
Input Operator ID:
[+] AUTHORIZATION ACCEPTED: Level 999 Admin.
[+] OMEGA_TOKEN active: XPLOIT-2026-OMEGA

Vault Unlock Code:
**************************************************
  VAULT SYSTEM CLEARED.
  All authentication layers bypassed successfully.
  Document your methodology and proceed to
  the next question.
**************************************************
```

7. Submission Checklist

- **chal_patched** — output of patch.py
- **patch.py** — binary patching script

- **solve.py** — key computation and solve script
- **writeup.pdf** — this document

Put all four files in a folder named after yourself, commit, and open a pull request.